Working With Dictionaries

In the previous lesson, we looked at how one can retrieve values using keys and how to add new entries or update existing ones. Those operations let us read and change information stored in a dictionary. In real programs, however, information does not stay the same forever. That is, a product may be discontinued, a task may be completed and removed from a list of pending tasks, or a saved setting in an application may no longer be needed.

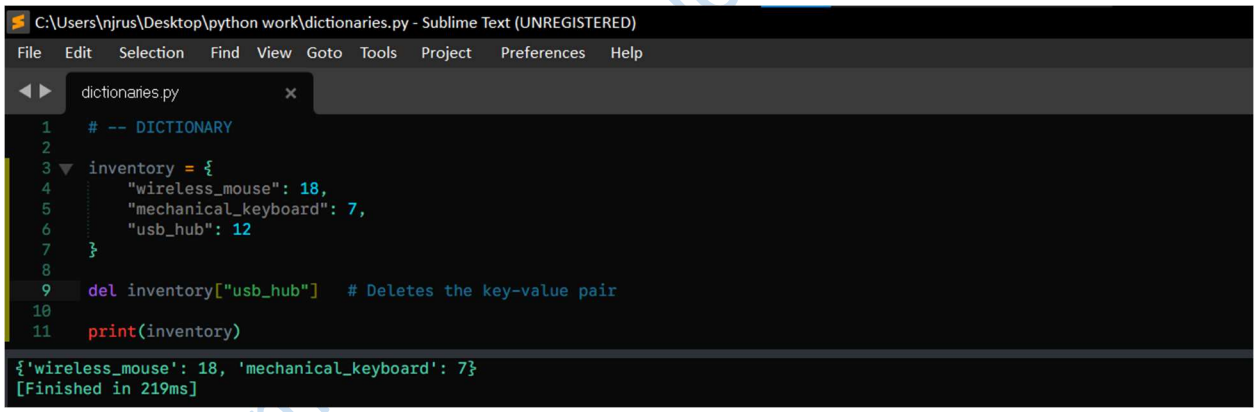
This lesson covers two related skills. First, we will look at how one can remove dictionary entries. Second, we will learn how to check whether a key exists before trying to use or remove it. Checking for a key is especially useful when the program cannot assume that every entry is present.

Removing Entries in Dictionaries

Dictionary entries can be removed using the **del statement**, **pop method** and **clear method**.

Using del statement

You can use the del statement when you know the key of the entry you want to remove. It deletes the key–value pair from the dictionary.

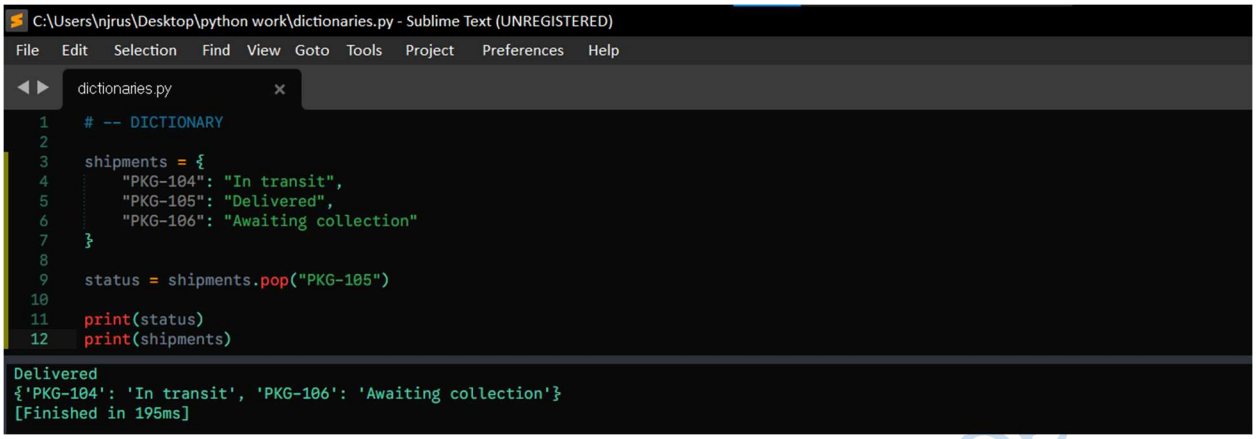


The entry with the key `"usb_hub"` is removed. The other entries remain unchanged.

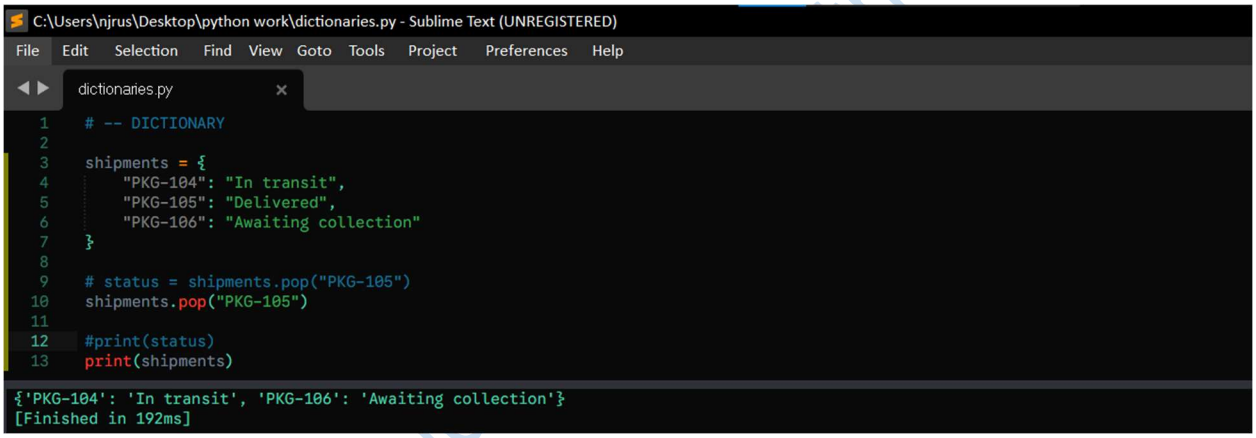
Note: if the key you are trying to delete is not in the dictionary, `del dict_name['your_key']` raises a KeyError.

Using the pop method

The `pop()` method removes an entry for a specified key and returns the value that was stored under that key.

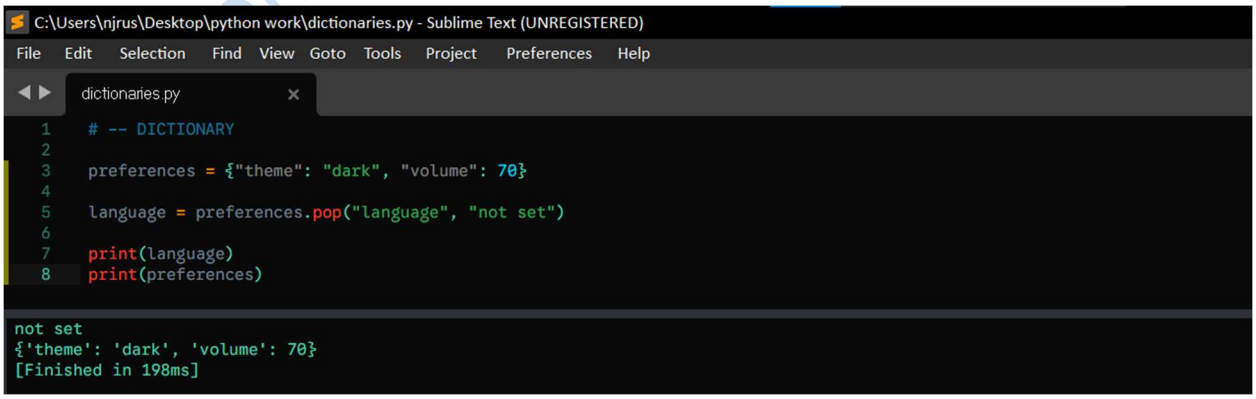


pop("PKG-105") removes that entry and returns "Delivered", which is assigned to status. However, storing the popped value is only useful when you need to use its former value. If this is not necessary, avoid storing the value and use the pop method as shown below.



It produces the same results. The only difference is if you would need the popped value later on.

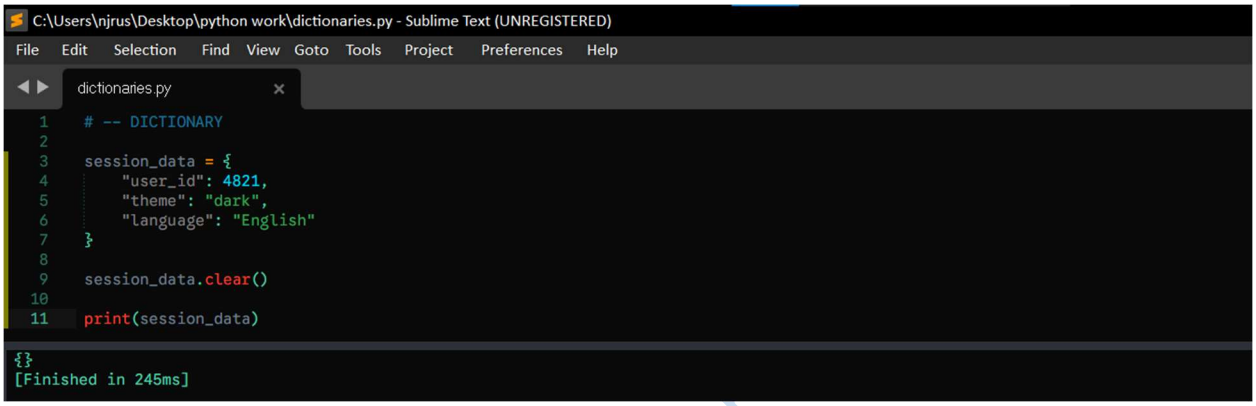
If a key might be missing, pop() can accept a second argument. This argument is returned when the key does not exist, and no KeyError is raised.



However, remember that, the default value is only returned when the key is absent. It does not create that key.

Using the clear method

This method is used when one wants to remove every entry from a dictionary. The dictionary itself still exists, but it becomes empty.

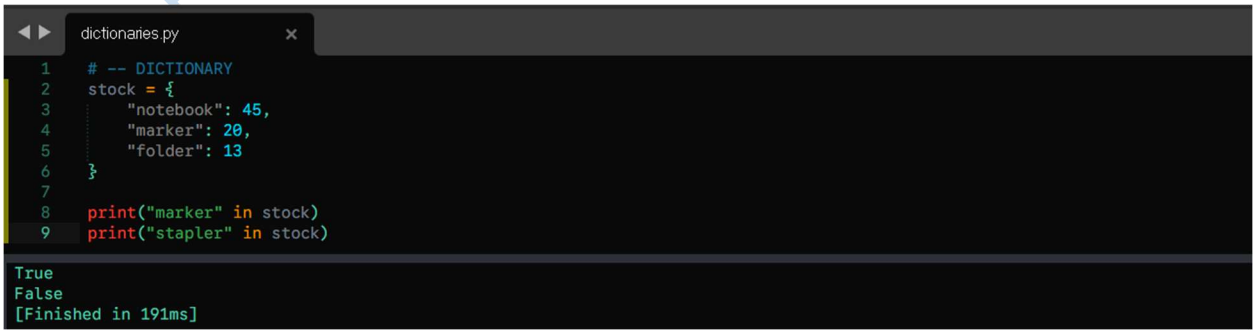


After clear(), session_data is an empty dictionary. You can add new entries to it later.

Note: The style you choose to remove your entries is based on what you need. del is a straightforward deletion. Use pop() when you need the removed value, and clear() when the entire dictionary should be emptied.

Checking Keys Using 'in' and 'not in'

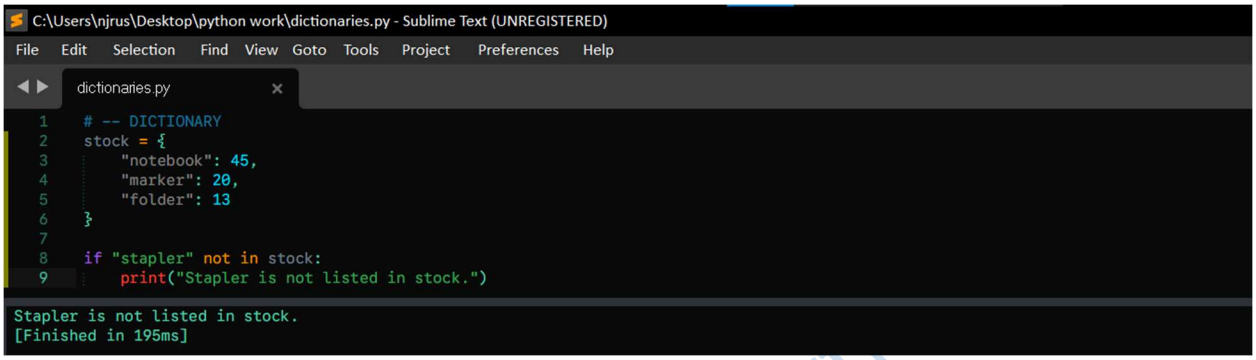
The in operator can be used to test whether a key is present in a dictionary. The result is a Boolean value, that is, True if the key exists and False if it does not.



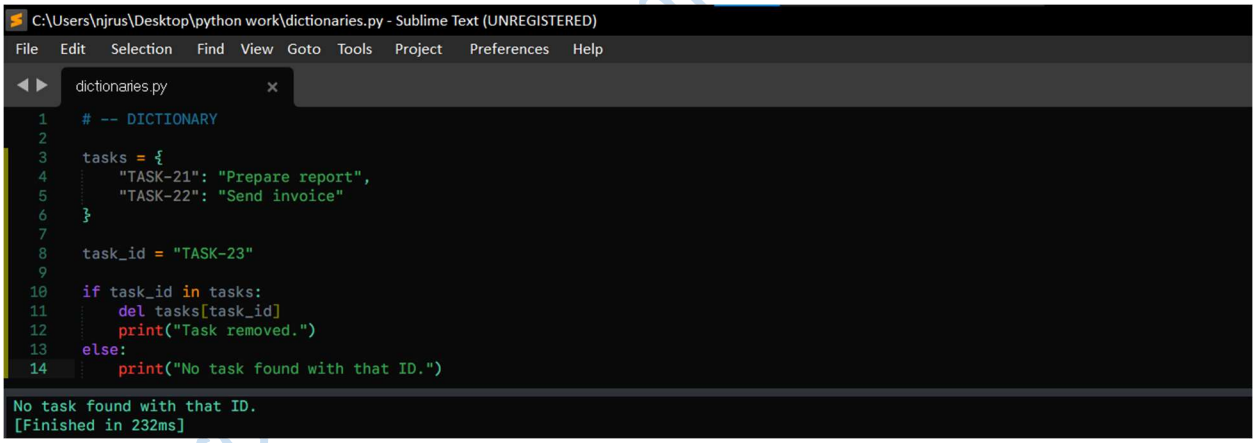
From Basics to Junior

The expression "marker" in stock checks the dictionary's keys. It does not check whether 20 or another value appears among the values.

On the other hand, the not in operator is used to test whether a key is absent. It is the logical opposite of in.



Checking for a key helps a program make decisions based on the information that is actually available. It is particularly useful before accessing or deleting an entry when that key may not exist.



Alternative: If you do not need a separate check, tasks.pop(task_id, None) can remove the key if it exists and return None if it does not. Use a different default if None could be a meaningful stored value.